Easy Fashion Limited

Software Engineer Technical Assessment

Assessment Overview

Develop a modern, responsive Fashion E-Commerce application along with a secure Management Dashboard.

This assessment is designed to evaluate your software engineering skills, including backend architecture, frontend development, authentication & authorization, database design, REST API development, clean coding practices, UI/UX implementation, security, and overall project architecture.

Submission Deadline

07 August 2026 (11:59 PM)

Preferred Technology Stack

Backend

- Node.js
- Express.js or Nests

Database

- PostgreSQL or MongoDB (Mongoose)

Database Access

Use any ORM or Query Builder:

- Sequelize

- TypeORM
- Prisma
- Knex
- Mongoose

Frontend

- Next.js

UI Framework

- Tailwind CSS
- Ant Design
- Material UI
- Bootstrap
- Or any modern CSS framework

Project Modules

The project consists of two applications:

1. Customer E-Commerce Website
2. Management Dashboard

Module 1 – Customer Website

Home Page

The landing page should include:

Hero Section

- Modern responsive design
- Animated banner or carousel
- Fashion promotional banners
- Smooth animations

Summary Section

Display attractive summary cards showing:

- Total Categories
- Total Products
- Available Sizes
- Available Styles

Product Listing

Display products in responsive card view.

Each product card should include:

- Product Image
- Product Name
- Category
- Style
- Available Sizes
- Price
- Add to Cart button

Product Filtering

Allow filtering by:

- Category
- Size
- Style

Products should update dynamically based on selected filters.

Product Details

Clicking a product should open a details page displaying:

- Multiple Product Images
- Product Description
- Category
- Available Sizes
- Available Styles
- Price
- Quantity Selector
- Add to Cart

Shopping Cart

Implement:

- Add Item
- Remove Item
- Update Quantity
- Price Calculation
- Grand Total

Checkout / Order

Collect:

- Customer Name
- Phone Number
- Shipping Address

Generate an Order successfully.

Footer

Include:

- Company Information
- Contact Information
- Social Media Links
- Copyright

Module 2 – Management Dashboard

Develop a secure and responsive Management Dashboard.

Authentication & Authorization

Implement a secure authentication and authorization system.

User Registration

Allow users to register using:

- Full Name
- Email Address
- Phone Number (Optional)
- Password

Requirements

- Email must be unique.
- Passwords must be securely hashed using bcrypt.
- Validate all input fields.
- Return proper validation errors.
- Newly registered users should receive the **Customer** role by default.

User Login

Users should be able to log in using email and password.

After successful login generate:

- JWT Access Token
- JWT Refresh Token

Requirements

- Store Refresh Token securely (hashed if persisted).
- Implement Access Token expiration.
- Implement Refresh Token expiration.
- Implement Refresh Token API.

Required APIs

- Register
- Login
- Refresh Token
- Logout
- Get Current User Profile

Social Authentication

Implement OAuth Login using:

- Google
- Facebook

Requirements:

- Existing users should be logged in automatically.
- New users should be created automatically.
- Return JWT Access Token and Refresh Token after successful authentication.

Dashboard Authentication

Only authenticated dashboard users should access the Management Dashboard.

The system must contain one default **Super Admin** account seeded directly into the database.

The Super Admin must be able to:

- Login
- Create Dashboard Users
- View User List
- View User Details
- Update User Information
- Activate / Deactivate Users
- Assign User Roles

Authorization (RBAC)

Implement Role-Based Access Control.

Roles

- Super Admin
- Admin
- Manager
- Customer

Permissions

Super Admin

- Full System Access
- Manage Dashboard Users
- Assign Roles & Permissions
- Manage Products
- Manage Categories
- Manage Sizes

- Manage Styles
- Manage Orders
- View Dashboard Reports

Admin

- Manage Products
- Manage Categories
- Manage Sizes
- Manage Styles
- View Users
- Manage Orders

Manager

- View Dashboard
- View Products
- Manage Orders
- Update Order Status

Customer

- Register
- Login
- Browse Products
- Add to Cart
- Place Orders
- View Own Orders
- Update Own Profile

All dashboard APIs must be protected using JWT Authentication and Role Guards.

Security Best Practices

Implement:

- JWT Authentication
- Password Hashing (bcrypt)
- Refresh Token Rotation (Bonus)
- Route Guards / Middleware
- Protected APIs
- Proper HTTP Status Codes
- Input Validation
- Centralized Error Handling
- CORS Configuration
- Environment Variables for Secrets

Bonus Security Features

- Email Verification
- Forgot Password
- Reset Password
- Account Activation
- Rate Limiting
- Login Attempt Protection
- Audit Log for Login Activity

Dashboard

Display dashboard summary cards:

- Total Users
- Total Categories
- Total Products
- Total Orders

Category Management

Implement full CRUD operations.

Product Management

Implement full CRUD operations.

Each product should contain:

- Product Name
- Category
- Style
- Size
- Description
- Price
- Multiple Product Images

Size Management

Implement full CRUD operations.

Style Management

Implement full CRUD operations.

Order Management

Display:

- Customer Information
- Ordered Products
- Quantity
- Total Amount
- Order Status

Allow updating Order Status.

User Management

Display:

- User List
- User Details

Super Admin should be able to create and manage dashboard users.

API Requirements

Develop clean REST APIs following best practices.

Include:

- Validation
- Authentication
- Authorization
- Error Handling
- Pagination
- Search
- Filtering

Database Design

Design a normalized relational database including:

- Users
- Roles
- Categories
- Products
- Sizes
- Styles
- Orders
- Order Items

Use proper relationships, foreign keys, indexes, and constraints.

Code Quality Expectations

Evaluation will consider:

- Clean Architecture
- Modular Design
- Folder Structure
- Reusable Components
- SOLID Principles
- Clean Code
- API Design
- Security
- Performance
- Responsive UI
- Git Commit History

Git Workflow

1. Accept the GitHub Collaboration invitation.
2. Clone the repository.
3. Complete the project according to the requirements.
4. Make meaningful Git commits.
5. Push all code before the submission deadline.

Submission Checklist

Before submitting, ensure that:

- Source code has been pushed to GitHub.
- The project runs successfully without errors.
- A complete README file is included.
- Installation steps are documented.
- Environment variables are documented.
- Database migration, schema, or seed files are included.

Submission Confirmation

After completing the assessment, reply to the assignment email with:

- Full Name
- GitHub Username
- Submission Date & Time
- Confirmation that all source code has been pushed successfully.
- Brief summary of the completed work.
- Any assumptions, limitations, or additional features implemented.

We wish you the very best and look forward to reviewing your technical solution.

Easy Fashion Limited